A Case Study Report
on

ONLINE SHOPPING SYSTEM

is submitted in partial fulfillment of requirements for
the award of CIE of

SOFTWARE ENGINEERING (22CSC21N)

in
INFORMATION TECHNOLOGY

Submitted by
Vishnu Teja Karampuri (1601-24-737-112)

Course Instructor:
B Swathi
Assistant Professor, Dept. of IT
DEPARTMENT OF INFORMATION TECHNOLOGY

CHAITANYA BHARATHI INSTITUTE OF TECHNOLOGY (A)
(Affiliated to Osmania University; Accredited by NBA, NAAC, ISO)
Kokapet (V), Gandipet (M), Hyderabad – 500075
Website: www.cbit.ac.in





CHAITANYA BHARATHI
INSTITUTE OF TECHNOLOGY
Autonomous Institute | Affiliated to Osmania University

CERTIFICATE

This is to certify that the course-end project work entitled **"Online Shopping System"** is submitted by **Vishnu Teja Karampuri (1601-24-737-112)**, in partial fulfillment of the requirements for the award of **CIE Marks of Software Engineering (22CSC21N) of B.E, V Semester, Information Technology, to Chaitanya Bharathi Institute of Technology (Autonomous) affiliated to Osmania University, Hyderabad.**

This is a record of bonafide work carried out by the student under my supervision and guidance. The results embodied in this report have not been submitted to any other University or Institute for the award of any Degree or Diploma.

Course Instructor

B Swathi

Assistant Professor, Dept. of IT

Vishnu Teja Karampuri: 1601-24-737-112

ABSTRACT

Retail buyers now expect to browse, compare and pay for goods from a phone or laptop at any hour. Small and mid-sized sellers, however, still depend on walk-in counters, telephone orders and spreadsheets, which leads to **stock mismatches, lost orders, slow fulfilment and no consolidated view of sales**.

This case study documents the analysis and design of the **Online Shopping System (OSS)**, a role-based web platform in which Customers browse and purchase products, Vendors list items and fulfil orders, and Administrators govern users, promotions and reporting. All requirements are derived from the approved Software Requirements Specification (SRS v1.0), which defines 35 functional requirements across eight feature areas and nine non-functional requirements covering performance, security and availability.

The report selects the **Incremental process model**, justifies it against alternatives, and assesses technical, economic, operational and schedule feasibility. The system is designed as a **three-tier architecture**: a responsive single-page front end, a stateless RESTful API (Node.js/Express with JWT-based RBAC) and a MongoDB document database. Analysis and design are communicated through use case, class, sequence, activity, data-flow and entity-relationship diagrams, together with collection schemas and key algorithms such as stock-safe checkout and coupon validation.

A **BlueCart** interface prototype of 26 screens, covering the customer storefront, vendor console and administrator panel, was designed to validate the requirements visually. A test suite of 30 designed test cases is traceable to every feature area. The report closes with advantages, limitations and a roadmap for future releases.

Keywords: Online Shopping System, E-commerce, SRS, Incremental Model, UML, Role-Based Access Control, JWT, MongoDB, Shopping Cart, Payment Gateway, Prototype.

Table of Contents

CHAPTER 1: INTRODUCTION 1
- 1.1 Overview 1
- 1.2 Problem Statement 1
- 1.3 Objectives 1
- 1.4 Scope of the Project 1
- 1.5 Organization of the Report 2

CHAPTER 2: LITERATURE REVIEW & BACKGROUND 2
- 2.1 Existing Systems 2
- 2.2 Limitations of Existing Systems 2
- 2.3 Proposed System: BlueCart OSS 2

CHAPTER 3: SYSTEM REQUIREMENTS ANALYSIS 3
- 3.1 User Classes and Characteristics 3
- 3.2 Functional Requirements (Traceability Matrix) 3
- 3.3 Non-Functional Requirements 4
- 3.4 Business Rules 4
- 3.5 Hardware and Software Requirements 4

CHAPTER 4: SDLC MODEL & FEASIBILITY STUDY 5
- 4.1 Selection of the SDLC Model 5
- 4.2 Feasibility Study 5
- 4.3 Risk Analysis 6

CHAPTER 5: SYSTEM DESIGN 6
- 5.1 System Architecture (Three-Tier Topology) 6
- 5.2 Layered Application Design and API Surface 7
- 5.3 Use Case Modeling 7
- 5.4 Class Diagram 8
- 5.5 Database Design 8
- 5.6 Sequence Diagram: Checkout and Payment 9
- 5.7 Activity Diagram: Purchase Workflow 9
- 5.8 Data Flow Diagram (Level 0) 10
- 5.9 Order Lifecycle 10

CHAPTER 6: IMPLEMENTATION, MODULES & ALGORITHMS 10
- 6.1 Technology Stack 10
- 6.2 Project Structure 10
- 6.3 Module Description 11
- 6.4 Key Algorithms 11
- 6.5 Prototype Description and Design Language 12

CHAPTER 7: TESTING & VERIFICATION 12
- 7.1 Testing Strategy 12
- 7.2 Designed Test Cases 12
- 7.3 Requirements Verification Matrix 13

CHAPTER 8: RESULTS, USER INTERFACES & WORKFLOWS 13
- 8.1 Customer Authentication Screens 13
- 8.2 Home, Category and Search 14
- 8.3 Product Details and Reviews 15
- 8.4 Cart and Wishlist 16
- 8.5 Checkout, Payment and Confirmation 16
- 8.6 Orders, Profile and Information Pages 17
- 8.7 Vendor Console 18
- 8.8 Administrator Panel 18
- 8.9 Summary of Results 21

CHAPTER 9: ADVANTAGES, LIMITATIONS, FUTURE ENHANCEMENTS & CONCLUSION 21
- 9.1 Advantages 21
- 9.2 Limitations 21
- 9.3 Future Enhancements 21
- 9.4 Conclusion 21

REFERENCES 22

List of Figures

Figure 5.1 Three-tier architecture of the OSS with external services 6
Figure 5.2 Use case diagram of the Online Shopping System 7
Figure 5.3 Class diagram: core domain model 8
Figure 5.4 Entity-relationship diagram of the OSS 8
Figure 5.5 Sequence diagram for checkout and payment 9
Figure 5.6 Activity diagram of the purchase workflow 9
Figure 5.7 Level-0 data flow diagram of the OSS 10
Figure 8.1 Login page 14
Figure 8.2 Customer registration page 14
Figure 8.3 Storefront home page with featured products 14
Figure 8.4 Product listing with filters and sorting 15
Figure 8.5 Category page (Audio) 15
Figure 8.6 Search results page 15
Figure 8.7 Product details page 15
Figure 8.8 Ratings and reviews page 16
Figure 8.9 Shopping cart with coupon and order summary 16
Figure 8.10 Wishlist page 16
Figure 8.11 Checkout: shipping address and delivery method 17
Figure 8.12 Secure payment page 17
Figure 8.13 Order confirmation page 17
Figure 8.14 My Orders page with order status 17
Figure 8.15 User profile page 18
Figure 8.16 Contact Us page 18
Figure 8.17 About Us page 18
Figure 8.18 Vendor console with orders to fulfil 18
Figure 8.19 Administrator login 19
Figure 8.20 Coupons and promotions 19
Figure 8.21 Administrator dashboard 19
Figure 8.22 Product management 20
Figure 8.23 Order management 20
Figure 8.24 Customer management 20
Figure 8.25 Inventory management 20
Figure 8.26 Reports and analytics 20

List of Tables

Table 2.1 Existing practice versus the proposed OSS 2
Table 3.1 User classes of the OSS 3
Table 3.2 Functional requirements of the OSS 3
Table 3.3 Non-functional requirements 4
Table 3.4 Deployment requirements 4
Table 4.1 Comparison of candidate process models 5
Table 4.2 Increment plan 5
Table 4.3 Indicative cost estimate for a pilot deployment 5
Table 4.4 Principal project risks 6
Table 5.1 Principal REST endpoints 7
Table 5.2 Use case specification UC-04: Checkout and Payment 7
Table 5.3 MongoDB collections (database: oss_db) 8
Table 5.4 Order states and permitted transitions 10
Table 6.1 Proposed implementation stack 10
Table 6.2 Functional modules of the OSS 11
Table 6.3 BlueCart design tokens 12
Table 7.1 Test levels 12
Table 7.2 Designed test cases for the OSS 12
Table 7.3 Coverage of feature areas by test cases 13
Table 8.1 Prototype coverage against the SRS 21

CHAPTER 1: INTRODUCTION

1.1 Overview

Shopping has gradually moved from physical bazaars to digital storefronts. A modern customer expects to search a catalogue, read honest reviews, compare prices, apply a discount code, pay securely and track the parcel, all without speaking to anyone. Behind this simple experience sits a tightly coordinated set of services: a product catalogue, an inventory ledger, a cart, a payment link, an order pipeline and a reporting layer.

Smaller retailers rarely have such infrastructure. Orders arrive through phone calls and chat messages, stock is tracked on paper or in a spreadsheet, and the owner learns what sold well only at the end of the month. The **Online Shopping System (OSS)** studied in this report is designed to close that gap. It is a centralised, role-based web application that manages the whole life cycle of an online retail operation, from listing a product to delivering it and analysing the sale.

The study is based on the approved SRS (Version 1.0, July 2026), which follows the IEEE Std 830-1998 structure. The SRS was converted into design artefacts, a test plan and a visual prototype, so that the same requirement can be followed from the document to a diagram, to a test case and finally to a screen.

1.2 Problem Statement

Retail businesses that still rely on manual or loosely connected tools face four recurring problems:

- **Inventory mismatch:** the same item can be promised to two buyers because stock is updated by hand, which leads to cancellations and unhappy customers.
- **Slow, error-prone ordering:** phone or in-person orders need a human to note down items, calculate totals and confirm payment, so mistakes in price and quantity are common.
- **Fragmented responsibilities:** vendors, store staff and owners use different notebooks or files, so nobody has one authoritative view of products, orders and customers.
- **No decision support:** without structured sales and stock data, restocking and promotion decisions are taken on instinct rather than evidence.

1.3 Objectives

- Replace phone and counter ordering with a **self-service storefront** where customers can browse, search, filter, manage a cart and pay online.
- Give vendors a **console to list products, maintain stock and fulfil orders** for their own catalogue only.
- Give administrators one **consolidated panel** for users, vendors, coupons, reviews and sales reports.
- Guarantee **data integrity and secure access** through JWT authentication, bcrypt hashing, role-based access control and PCI-DSS-compliant payment delegation.
- Keep the design **scalable and portable** through a stateless API tier and a containerisable back end.

1.4 Scope of the Project

In scope (Version 1.0):

- Three role-based workspaces: Customer storefront, Vendor console and Administrator panel.
- Registration, login, password reset, account lockout and role-based authorisation.
- Catalogue, SKU generation, stock control, search, filtering, ratings and reviews.
- Cart, wishlist, coupons, checkout, gateway payment, order tracking, cancellation and returns.
- Email/SMS notifications, sales and low-stock reports with CSV export.

Out of scope (candidates for later releases):

- Point-of-sale integration for physical stores and barcode scanners.
- Third-party marketplace syndication and logistics or fleet management.

- Multi-currency and multi-language support (English and a single currency in V1.0).

1.5 Organization of the Report

Chapter 2 reviews existing approaches and presents the proposed system; **Chapter 3** analyses functional and non-functional requirements; **Chapter 4** selects the SDLC model and studies feasibility; **Chapter 5** presents the architecture, UML models and database design; **Chapter 6** describes modules, algorithms and the prototype design language; **Chapter 7** documents the test plan; **Chapter 8** walks through the 26 prototype screens; and **Chapter 9** closes with advantages, limitations, future enhancements and the conclusion.

CHAPTER 2: LITERATURE REVIEW & BACKGROUND

2.1 Existing Systems

- **Manual and phone-based ordering:** the buyer calls or visits, the seller writes the order in a register and confirms stock from memory. There is no audit trail and no way to scale beyond a few dozen orders a day.
- **Spreadsheet and chat-app selling:** catalogues are shared as images or sheets and payments are collected through personal transfers. Records are scattered across devices, and a single overwritten cell can corrupt stock figures.
- **Hosted marketplace listing:** sellers rely on a large third-party marketplace. Reach is high, but commissions are heavy, the seller does not own the customer relationship, and branding, promotions and data access are limited by the marketplace.
- **Generic site builders:** template-based builders offer a storefront quickly but offer limited control over role-based workflows, custom reports or vendor-specific permissions.

2.2 Limitations of Existing Systems

- No real-time stock validation, so overselling is possible during busy periods.
- No unified cart-to-delivery workflow; payment, packing and tracking happen in separate tools.
- Weak access control: staff often share one login, so actions cannot be traced to a person.
- Reports are produced by hand and arrive too late to guide restocking or discount decisions.
- Sensitive payment details may be handled informally, which is a compliance and trust risk.

2.3 Proposed System: BlueCart OSS

The proposed OSS is a single, self-contained web product that supports three roles under one login system. The cart and checkout logic runs on the server, so a price or quantity cannot be altered from the browser. Stock is decremented only when payment succeeds, and every administrative action is written to an immutable audit trail. Payment cards are never stored; the gateway handles all card data.

Table 2.1: *Existing practice versus the proposed OSS*

Aspect	Existing practice	Proposed OSS
Ordering	Phone, counter or chat	Self-service web checkout with gateway payment
Stock accuracy	Manual, delayed updates	Automatic decrement and overselling check at checkout
Access control	Shared logins	JWT with role-based access (Admin, Vendor, Customer)
Order visibility	Ask the seller	Live status timeline and history for the customer
Promotions	Ad hoc price changes	Coupons with validity and usage limits
Reporting	Monthly manual tallies	Date-range sales and low-stock reports with CSV export
Security	Informal handling	bcrypt, TLS 1.2+, input sanitisation, audit trail

CHAPTER 3: SYSTEM REQUIREMENTS ANALYSIS

3.1 User Classes and Characteristics

Table 3.1: *User classes of the OSS*

User class	Responsibilities	Characteristics
Administrator	Manages vendors, users, coupons, reviews and system-wide reports	Technically proficient; needs access to all modules
Vendor / Seller	Lists products, maintains stock, fulfils orders, sets own discounts	Moderate proficiency; works mainly in catalogue and orders
Customer	Browses, buys, tracks orders, reviews products	Mixed skill; expects a simple, mobile-friendly interface

3.2 Functional Requirements (Traceability Matrix)

The 35 functional requirements below are taken from SRS Chapter 4. Each one is mapped to the prototype screen that demonstrates it (Chapter 8) and to at least one test case (Chapter 7).

Table 3.2: *Functional requirements of the OSS*

ID	Module	Requirement	Priority	Prototype screen
FR-01	Authentication	Customer self-registration with an email address and password	High	Registration
FR-02	Authentication	Login with registered email/username and password	High	Login
FR-03	Authentication	Issue a signed JWT with a configurable expiry on successful login	High	Login
FR-04	Authentication	Reject protected API calls that carry no valid JWT	High	Login
FR-05	Authentication	Enforce role-based access control (Admin, Vendor, Customer)	High	Admin Login
FR-06	Authentication	Password reset through a time-limited emailed link	Medium	Login
FR-07	Authentication	Lock an account for 15 minutes after 5 consecutive failed logins	Medium	Admin Login
FR-08	Catalog	Vendor creates a listing with name, description, category, price, stock and images	High	Product Mgmt
FR-09	Catalog	Auto-generate a unique SKU in a configurable format	High	Product Mgmt
FR-10	Catalog	Search and filter by category, price range and rating	High	Listing, Search
FR-11	Catalog	Decrement stock on order placement; block orders above available stock	High	Inventory
FR-12	Catalog	Vendor edits price, description and stock levels	Medium	Product Mgmt
FR-13	Catalog	Administrator soft-deletes (deactivates) a listing and keeps order history	Medium	Product Mgmt
FR-14	Cart	Add products with a chosen quantity to a persistent cart	High	Cart
FR-15	Cart	Change quantities or remove items before checkout	High	Cart
FR-16	Cart	Apply a valid coupon and recalculate the order total	Medium	Cart
FR-17	Cart	Block checkout when any quantity exceeds stock at that moment	High	Checkout
FR-18	Cart	Choose shipping address and shipping method during checkout	High	Checkout
FR-19	Cart	Pay through an integrated gateway before the order is confirmed	High	Payment
FR-20	Orders	Generate a unique order ID and confirmation after payment	High	Confirmation
FR-21	Orders	Vendor updates fulfilment status (Processing, Shipped, Delivered)	High	Vendor Console
FR-22	Orders	Customer views live order status and history	High	My Orders
FR-23	Orders	Customer requests cancellation before the order is shipped	Medium	My Orders
FR-24	Orders	Customer raises a return request inside the return window	Medium	My Orders

ID	Module	Requirement	Priority	Prototype screen
FR-25	Reviews	Purchaser submits a 1–5 star rating and written review	Medium	Reviews
FR-26	Reviews	Compute and show the average rating per product	Medium	Details, Reviews
FR-27	Reviews	Administrator removes a review that breaks content rules	Low	Reviews
FR-28	Promotions	Administrator defines coupons (percentage/flat, validity, usage limit)	Medium	Coupons
FR-29	Promotions	Reject expired or usage-exhausted coupons at checkout	Medium	Coupons, Cart
FR-30	Promotions	Vendor configures a time-bound discount on own listings	Low	Vendor Console
FR-31	Notifications	Email the customer on order confirmation	Medium	Confirmation
FR-32	Notifications	Notify on fulfilment status changes when enabled	Low	Profile
FR-33	Reporting	Sales summary for a date range, filterable by vendor/category	Medium	Reports
FR-34	Reporting	Low-stock report using a configurable threshold	Medium	Inventory
FR-35	Reporting	Export generated reports as CSV	Low	Reports

3.3 Non-Functional Requirements

Table 3.3: *Non-functional requirements*

ID	Category	Requirement
NFR-01	Performance	95% of API requests answered within 500 ms for up to 500 concurrent users
NFR-02	Performance	Catalogue of at least 50,000 products without measurable query slowdown
NFR-03	Performance	Checkout, from cart validation to payment confirmation, within 5 seconds under normal load
NFR-04	Security	Passwords hashed with bcrypt, cost factor of at least 10
NFR-05	Security	All traffic over TLS 1.2 or higher
NFR-06	Security	JWT access tokens expire within 24 hours; refresh tokens are revocable
NFR-07	Security	Input sanitisation against NoSQL injection and XSS (OWASP Top Ten)
NFR-08	Security	No raw card data stored; PCI-DSS compliant gateway used
NFR-09	Auditability	Admin and vendor actions logged to an immutable audit trail
Quality	Availability	Target 99.5% uptime excluding planned maintenance
Quality	Maintainability / Portability	Separated routes, controllers and models; Docker-ready back end
Quality	Usability / Scalability	Core task in 3 interactions; stateless API allows horizontal scaling

3.4 Business Rules

- Only an Administrator may create or deactivate vendor accounts.
- A Vendor may manage only products and orders attached to their own catalogue.
- A Customer can only view a confirmed order, apart from requesting cancellation or a return.
- Confirmed orders are immutable except through a logged Administrator or Vendor override.
- Deletions are soft deletes with a 30-day recovery window; transaction records are kept for 7 years.

3.5 Hardware and Software Requirements

Table 3.4: *Deployment requirements*

Component	Specification
Server hardware	Minimum 2 vCPU, 4 GB RAM, 10 GB SSD (cloud VM or container host)
Client hardware	Any desktop, laptop, tablet or smartphone (viewports 360 px to 1920 px)
Client software	Latest two major versions of Chrome, Firefox, Edge or Safari
Server software	Linux, Node.js LTS with Express.js, Docker for packaging

Component	Specification
Database	MongoDB 6.0+ (self-hosted or Atlas cluster)
External services	Payment gateway (Razorpay/Stripe), email/SMS gateway (SendGrid/Twilio)

CHAPTER 4: SDLC MODEL & FEASIBILITY STUDY

4.1 Selection of the SDLC Model

Five process models were compared for the OSS. The SRS already ranks every requirement as High, Medium or Low, the modules are loosely coupled, and a working core (login, catalogue, cart, checkout) is useful long before reports and promotions are ready. These traits favour a staged delivery approach.

Table 4.1: *Comparison of candidate process models*

Model	Strength	Weakness for the OSS	Verdict
Waterfall	Clear documents and phases	Late feedback; changes to checkout or payment are costly	Rejected
Prototyping	Early user feedback on screens	Does not alone give a deliverable architecture	Used inside each increment
Incremental	Working core first; priorities map to increments	Needs a stable overall architecture	**Selected**
Spiral	Strong risk analysis	Heavy process for a small team	Rejected
Agile / Scrum	Very adaptive	Requires an on-site customer and continuous team ceremony	Partly borrowed

The **Incremental model with prototype-driven reviews** is therefore adopted. The architecture is fixed up front (three tiers, stateless API, document database), and the functionality is delivered in four increments ordered by SRS priority. The BlueCart prototype is reviewed at the end of each increment so that interface problems are caught before the next one begins.

Table 4.2: *Increment plan*

Increment	Weeks	Scope (SRS requirements)	Outcome
1: Core	1–3	Authentication, RBAC, catalogue, SKU, stock (FR-01 to FR-13)	Users can sign in; vendors can list products
2: Commerce	4–6	Cart, coupons at checkout, payment, orders (FR-14 to FR-24)	End-to-end purchase works
3: Engagement	7–8	Reviews, coupons admin, notifications (FR-25 to FR-32)	Trust and promotion features
4: Insight	9–10	Reports, CSV export, hardening, NFR verification (FR-33 to FR-35)	Release candidate

4.2 Feasibility Study

4.2.1 Technical feasibility

All required capabilities are available in mature open-source tooling. JavaScript is used on both client and server, which reduces the skills the team needs. MongoDB handles flexible product attributes, JWT supports stateless horizontal scaling, and the payment gateway removes the need to handle card data. The minimum infrastructure (2 vCPU, 4 GB RAM) is modest. The project is rated **technically feasible**.

4.2.2 Economic feasibility

Table 4.3: *Indicative cost estimate for a pilot deployment*

Item	Basis	Estimated cost (INR)
Application server (2 vCPU / 4 GB)	Cloud VM, per month	1,200 – 2,000
Database	MongoDB Atlas shared tier for pilot	0 – 800
Domain and TLS certificate	Per year (TLS via free CA)	700 – 1,000
Email/SMS gateway	Free tier for pilot volumes	0 – 500
Payment gateway	No setup fee; per-transaction charge	Variable (about 2% of sales)

Figures are indicative planning values and may vary by provider. Because development uses open-source software and the running cost is a few thousand rupees per month, the benefit of reduced manual effort and fewer stock errors outweighs the cost, and the project is **economically feasible**.

4.2.3 Operational, schedule and legal feasibility

- **Operational:** the three role workspaces mirror real jobs (shopper, seller, store manager), and the 3-interaction usability target keeps training minimal.
- **Schedule:** four increments over ten weeks fit within a semester project timeline with one week of slack.
- **Legal:** card handling is delegated to a PCI-DSS gateway, personal data collection is limited to what is needed, and records follow the 7-year retention rule.

4.3 Risk Analysis

Table 4.4: *Principal project risks*

Risk	Likelihood	Impact	Mitigation
Gateway not finalised (TBD-01)	Medium	High	Adapter interface; sandbox gateway for development
Overselling during peak load	Medium	High	Atomic stock update inside the checkout transaction
Credential attacks	Medium	High	bcrypt, lockout after 5 failures, TLS, audit logging
Scope creep (marketplace commission, TBD-02)	Medium	Medium	Defer to Version 2.0; freeze increment scope

CHAPTER 5: SYSTEM DESIGN

5.1 System Architecture (Three-Tier Topology)

The OSS uses a decoupled three-tier architecture. The browser renders the interface and talks only to the REST API; the API owns all business rules and is the only component allowed to reach the database or an external service.

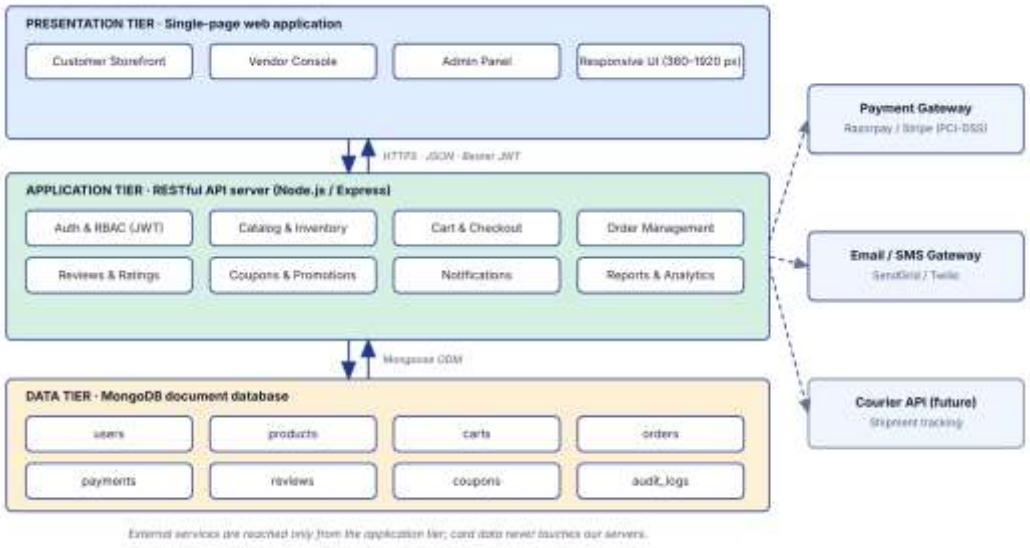


Figure 5.1: *Three-tier architecture of the OSS with external services*

The presentation tier hosts three role-specific workspaces in one responsive single-page application. The application tier is a stateless Node.js/Express service whose modules mirror the SRS feature areas. The data tier holds eight MongoDB collections. The payment and email gateways are reached only from the application tier, so card data never touches the platform's own servers.

5.2 Layered Application Design and API Surface

- **Routes** map HTTP endpoints to controllers and return consistent 400/401/403/404 errors.
- **Middleware** performs JWT verification, role checks, request validation and input sanitisation before any handler runs.
- **Services** hold business logic: stock reservation, coupon validation, order state transitions and report aggregation.
- **Models** (Mongoose schemas) define fields, unique indexes and references to other collections.

Table 5.1: *Principal REST endpoints*

Method and path	Role	Purpose
POST /api/auth/register · /login	Public	Create account; issue JWT
GET /api/products?category=&price=&rating=	Public	Search and filter catalogue (FR-10)
POST /api/products · PUT /api/products/:id	Vendor	Create or edit a listing (FR-08, FR-12)
DELETE /api/products/:id	Admin	Soft-delete a listing (FR-13)
POST /api/cart/items · PUT · DELETE	Customer	Manage cart lines (FR-14, FR-15)
POST /api/cart/coupon	Customer	Apply a coupon (FR-16)
POST /api/checkout · POST /api/payments/webhook	Customer / Gateway	Validate stock, start payment, confirm order
GET /api/orders · PUT /api/orders/:id/status	Customer / Vendor	History and fulfilment updates
POST /api/orders/:id/cancel · /return	Customer	Cancellation and return requests
POST /api/products/:id/reviews	Customer	Submit rating and review (FR-25)
POST /api/coupons	Admin	Define coupon (FR-28)
GET /api/reports/sales · /low-stock?format=csv	Admin / Vendor	Reports and CSV export (FR-33 to FR-35)

5.3 Use Case Modeling

Three actors interact with the system. Every actor must first authenticate; the remaining use cases are restricted by role. Customers handle discovery, purchase and feedback, vendors manage catalogue and fulfilment, and administrators manage governance and analytics.



Figure 5.2: *Use case diagram of the Online Shopping System*

Table 5.2: *Use case specification UC-04: Checkout and Payment*

Field	Description
Actor	Customer
Precondition	Customer is logged in and has at least one item in the cart
Main flow	1. Review cart and proceed. 2. Choose address and shipping method. 3. Choose payment method. 4. System validates stock for all items and redirects to the gateway. 5. On success, system creates the order, decrements stock and clears the cart.
Alternate flow	If any item is out of stock at validation, an error is shown and the customer adjusts the cart; if payment fails, the customer may retry.
Postcondition	Order saved as "Confirmed"; stock updated; confirmation email dispatched

5.4 Class Diagram

The domain model is organised around an abstract User that specialises into Customer, Vendor and Administrator. A Customer owns a Cart and places Orders; each Order contains OrderItems that reference Products; a Payment settles each Order; Reviews link Customers to Products; and Coupons can be redeemed against carts and orders.

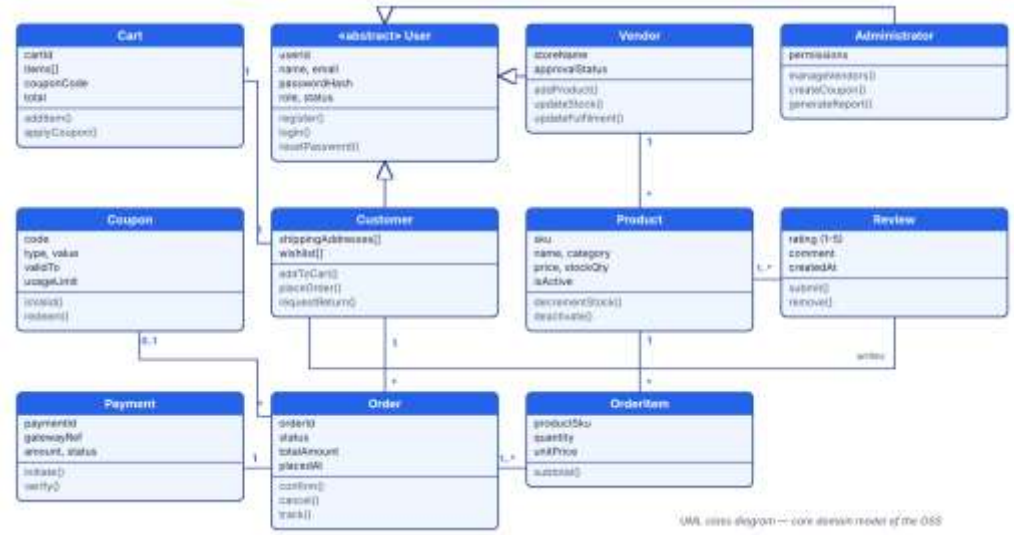


Figure 5.3: *Class diagram: core domain model*

5.5 Database Design

5.5.1 Entity-relationship model

A user may own one cart and many orders. Products belong to a vendor and may receive many reviews. An order contains many order items and is settled by one payment; a coupon can be referenced by many orders.

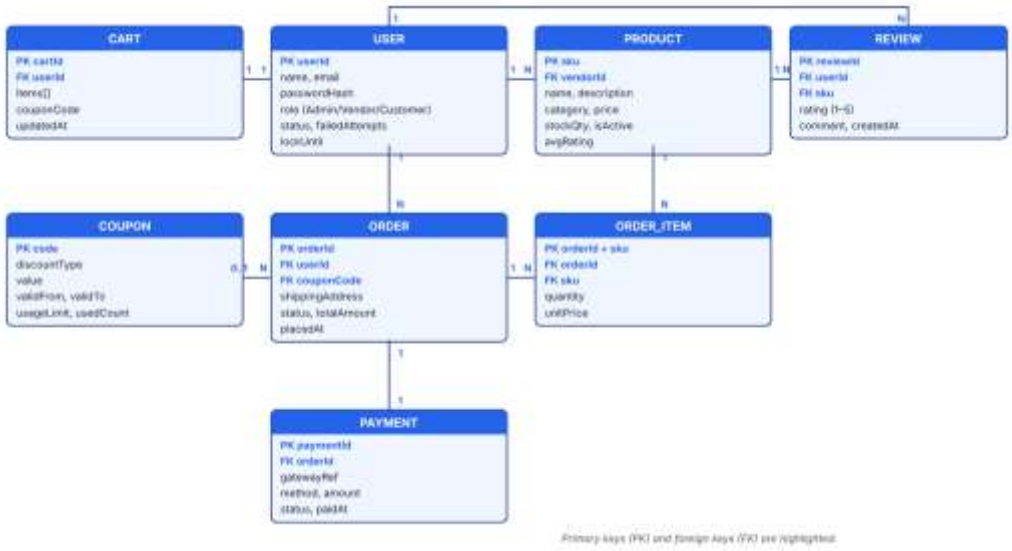


Figure 5.4: *Entity-relationship diagram of the OSS*

5.5.2 Collections and schema

Table 5.3: *MongoDB collections (database: oss_db)*

Collection	Key fields	Purpose
users	_id, name, email (unique), passwordHash (bcrypt), role, status, failedAttempts, lockUntil	Accounts for all roles
products	_id, sku (unique), vendorId, name, description, category, price, stockQty, images[], isActive, avgRating	Catalogue and stock
carts	_id, userId, items[{sku, qty}], couponCode, updatedAt	Persistent cart

Collection	Key fields	Purpose
orders	_id, orderId (unique), userId, items[{sku, qty, unitPrice}], address, shipping, totalAmount, status, placedAt	Confirmed orders
payments	_id, orderId, gatewayRef, method, amount, status, paidAt	Gateway references (no card data)
reviews	_id, userId, sku, rating (1–5), comment, createdAt	Ratings and reviews
coupons	_id, code (unique), type, value, validFrom, validTo, usageLimit, usedCount	Promotions
audit_logs	_id, actorId, action, entity, before, after, at	Immutable audit trail

Unique indexes on email, sku, orderId and coupon code reject duplicates at database level, independent of application code. Deleted products and orders carry an isActive/deletedAt flag so that history is preserved and recovery is possible for 30 days.

5.6 Sequence Diagram: Checkout and Payment

The sequence below shows how stock validation, gateway payment and order creation are ordered. The order is created only after the gateway's webhook confirms payment, which prevents orders without money and avoids decrementing stock for abandoned payments.

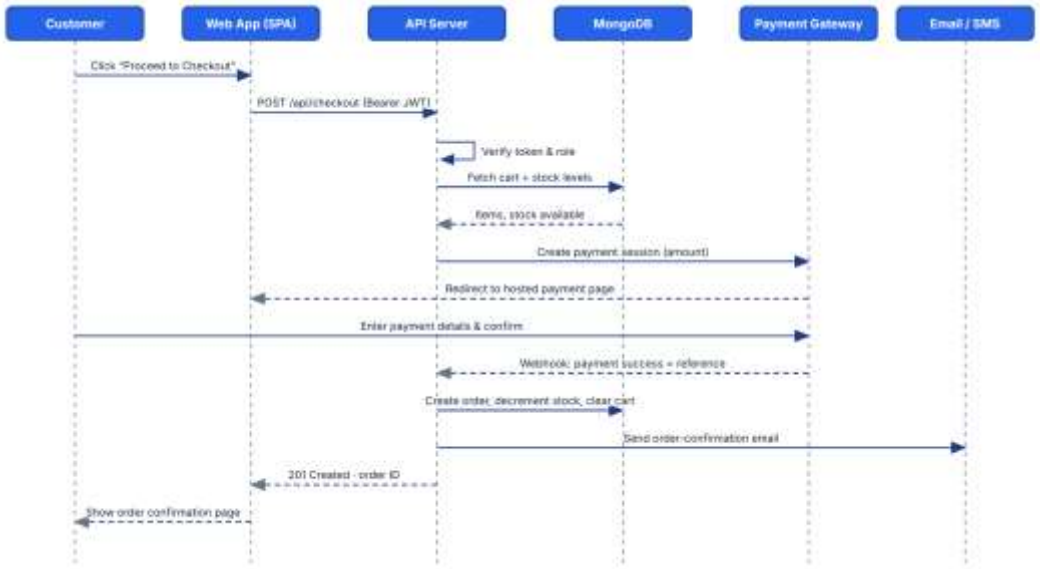


Figure 5.5: *Sequence diagram for checkout and payment*

5.7 Activity Diagram: Purchase Workflow

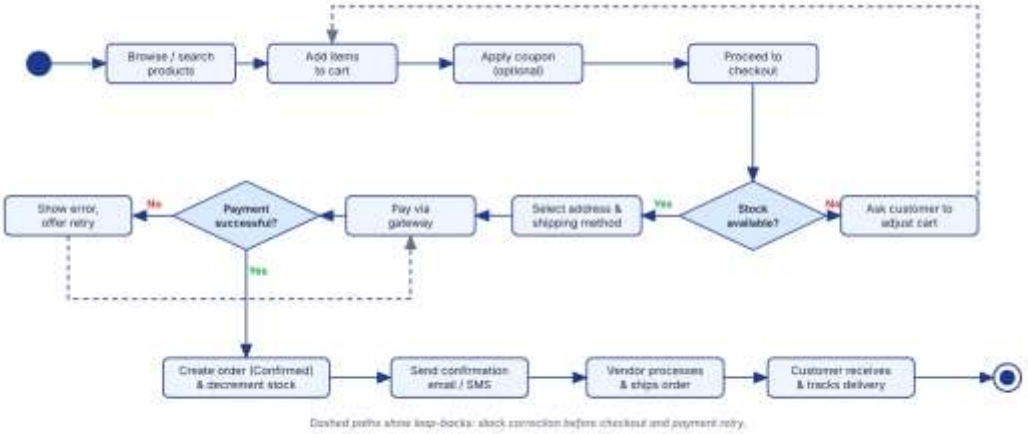


Figure 5.6: *Activity diagram of the purchase workflow*

Two feedback loops are important: a customer whose cart exceeds stock is returned to the cart rather than allowed to pay, and a failed payment returns to the payment step without touching stock.

5.8 Data Flow Diagram (Level 0)



Figure 5.7: *Level-0 data flow diagram of the OSS*

5.9 Order Lifecycle

Table 5.4: *Order states and permitted transitions*

State	Entered when	Next states	Changed by
Confirmed	Payment succeeds	Processing, Cancelled	System
Processing	Vendor starts packing	Shipped, Cancelled	Vendor
Shipped	Handed to courier	Delivered	Vendor
Delivered	Courier confirms delivery	Return Requested	Vendor
Return Requested	Customer asks within return window	Returned, Delivered	Customer / Admin
Cancelled	Requested before Shipped	(final)	Customer / Admin

CHAPTER 6: IMPLEMENTATION, MODULES & ALGORITHMS

6.1 Technology Stack

Table 6.1: *Proposed implementation stack*

Layer	Technology	Reason for selection
Front end	React single-page app, responsive CSS	Component reuse across three role workspaces
Back end	Node.js (LTS), Express.js	Lightweight REST API with middleware pipeline
Database	MongoDB with Mongoose ODM	Flexible product attributes; schema validation and indexes
Security	bcrypt, jsonwebtoken, helmet, express-validator	Hashing, stateless auth, header hardening, input validation
Payments	Razorpay or Stripe (hosted checkout)	PCI-DSS scope stays with the gateway
Delivery	Docker, Nginx reverse proxy, TLS	Portable deployment and horizontal scaling

6.2 Project Structure

```
oss/
|-- server/                    (Node.js + Express)
|   |-- app.js                 entry point, CORS, route mounting
|   |-- middleware/ auth.js rbac.js validate.js audit.js
|   |-- models/    User Product Cart Order Payment Review Coupon AuditLog
|   |-- routes/    auth catalog cart checkout orders reviews coupons reports
|   `-- services/  stock.js coupon.js pricing.js notify.js report.js
`-- client/                    (React SPA)
    `-- src/ pages/ (storefront, vendor, admin)  components/  api/
```

6.3 Module Description

Table 6.2: *Functional modules of the OSS*

Module	Responsibilities	SRS reference
Authentication & Access Control	Registration, login, JWT issue/verify, password reset, 5-attempt lockout, RBAC checks on every protected route	4.1, NFR-04 to 06
Catalog & Inventory	Listings, SKU generation, search/filter, stock update, soft delete, low-stock detection	4.2
Cart & Checkout	Persistent cart, wishlist, coupon application, address and shipping choice, stock re-validation, payment initiation	4.3
Order Management	Order creation, unique order ID, status timeline, cancellation and return handling, vendor fulfilment	4.4
Reviews & Ratings	Verified-purchase reviews, rating aggregation, moderation	4.5
Discounts & Promotions	Coupon rules (type, value, validity, usage limit) and vendor time-bound discounts	4.6
Notifications	Email on order confirmation; optional email/SMS on status change	4.7
Reporting & Analytics	Sales summary, low-stock report, CSV export, dashboard metrics	4.8

6.4 Key Algorithms

6.4.1 Login with account lockout (FR-02, FR-07)

```
function login(email, password):
    user = users.findByEmail(email)
    if user is null: return error 401
    if user.lockUntil > now(): return error 423 "Account locked"
    if bcrypt.compare(password, user.passwordHash):
        user.failedAttempts = 0
        return jwt.sign({id: user.id, role: user.role}, SECRET, expiresIn 24h)
    user.failedAttempts += 1
    if user.failedAttempts >= 5: user.lockUntil = now() + 15 minutes
    return error 401
```

6.4.2 Stock-safe checkout (FR-11, FR-17, FR-19, FR-20)

```
function checkout(userId, addressId, shippingMethod):
    cart = carts.find(userId)
    for line in cart.items:                          # step 1: validate
        if products.stock(line.sku) < line.qty: return error "Insufficient stock"
    total = pricing.total(cart, shippingMethod)    # server-side price
    session = gateway.createSession(total)         # step 2: payment
    wait for webhook(session)
    if webhook.status == "paid":                   # step 3: atomic commit
        begin transaction
            for line in cart.items: products.decrement(line.sku, line.qty)
            order = orders.create(cart, address, total, status="Confirmed")
            carts.clear(userId)
        commit
        notify.email(userId, order)
        return order
```

6.4.3 Coupon validation (FR-16, FR-29) and rating update (FR-26)

```
function applyCoupon(cart, code):
    c = coupons.find(code)
    if c is null or now() < c.validFrom or now() > c.validTo: return error
    if c.usedCount >= c.usageLimit: return error "Coupon exhausted"
    discount = (c.type == "percent") ? cart.subtotal * c.value / 100 : c.value
    return min(discount, cart.subtotal)

function updateRating(sku, newRating):             # new average of n+1 ratings
    p = products.find(sku)
    p.avgRating = (p.avgRating * p.reviewCount + newRating) / (p.reviewCount + 1)
    p.reviewCount += 1
```

6.5 Prototype Description and Design Language

The prototype is a set of high-fidelity HTML/CSS screens that reproduce what the delivered application is intended to look like, built from the SRS rather than from running code. The **BlueCart** theme is a clean blue-and-white commerce style: a blue top bar with a persistent cart indicator (SRS 3.1), white content cards with soft borders, a dark navy sidebar for the vendor and admin consoles, and status pills in green, amber and red. All screens are drawn for a 1280–1920 px desktop viewport; layouts collapse to a single column at 360 px.

Table 6.3: *BlueCart design tokens*

Element	Specification
Primary colour	Blue (#2563EB) for actions, links and active states
Surfaces	White cards on a very light blue-grey background; navy (#0F1B3D) admin sidebar
Status colours	Green = confirmed/active, amber = pending/low stock, red = failed/suspended
Typography	Inter, sans-serif; 13 px body text, 20–28 px headings
Components	Rounded 10 px cards, filled primary buttons, outline secondary buttons, pill badges, data tables with sticky headers

CHAPTER 7: TESTING & VERIFICATION

7.1 Testing Strategy

The test plan follows the V-model idea of pairing each development level with a test level. Test cases were written directly from the SRS so that every functional requirement has at least one verifying case. The cases below are specified with their inputs and expected outcomes and are meant to be executed against the implementation of each increment; the status column records the planned test level.

Table 7.1: *Test levels*

Level	Focus	Planned tools
Unit	Pricing, coupon, rating and lockout functions	Jest
Integration / API	Endpoints, RBAC and database rules	Supertest, Postman collections
System / UI	Complete purchase and fulfilment journeys	Playwright end-to-end scripts
Non-functional	Load, security headers, injection attempts	k6, OWASP ZAP
Acceptance	Review of each prototype screen against its requirement	Checklist with instructor

7.2 Designed Test Cases

Table 7.2: *Designed test cases for the OSS*

ID	Module	Test scenario	Expected result	Requirement	Level
TC-01	Auth	Register with a new email and valid password	Account created; 201	FR-01	API
TC-02	Auth	Register with an email that already exists	Rejected; duplicate error	FR-01	API
TC-03	Auth	Login with valid credentials	JWT returned with expiry	FR-02, 03	API
TC-04	Auth	Call a protected endpoint without a token	401 Unauthorized	FR-04	API
TC-05	Auth	Customer calls an admin-only endpoint	403 Forbidden	FR-05	API
TC-06	Auth	Five wrong passwords in a row, then correct one	Locked for 15 minutes	FR-07	API
TC-07	Auth	Request and use a password-reset link	Password changed; link expires	FR-06	System
TC-08	Catalog	Vendor creates a listing with all mandatory fields	Product saved with auto SKU	FR-08, 09	API
TC-09	Catalog	Create listing with missing price	Validation error	FR-08	Unit
TC-10	Catalog	Filter by category, price range and rating	Only matching products returned	FR-10	API

ID	Module	Test scenario	Expected result	Requirement	Level
TC-11	Catalog	Admin deactivates a product that has past orders	Hidden from store; history kept	FR-13	API
TC-12	Cart	Add item and change quantity	Cart total recalculated	FR-14, 15	System
TC-13	Cart	Apply valid coupon SAVE10	10% discount applied	FR-16	Unit
TC-14	Cart	Apply expired or exhausted coupon	Coupon rejected	FR-29	Unit
TC-15	Checkout	Checkout with quantity above stock	Blocked with stock error	FR-17	API
TC-16	Checkout	Complete payment in gateway sandbox	Order Confirmed; stock reduced	FR-11, 19, 20	System
TC-17	Checkout	Payment fails at the gateway	No order; stock unchanged; retry offered	FR-19	System
TC-18	Orders	Vendor moves order Processing to Shipped	Status saved; customer notified	FR-21, 32	System
TC-19	Orders	Customer views order history	Orders with live status shown	FR-22	System
TC-20	Orders	Cancel an order before it is shipped	Order Cancelled	FR-23	API
TC-21	Orders	Cancel an order that is already shipped	Rejected	FR-23	API
TC-22	Orders	Raise a return inside / outside window	Accepted / rejected	FR-24	API
TC-23	Reviews	Purchaser submits 4-star review	Saved; average rating updated	FR-25, 26	API
TC-24	Reviews	Non-purchaser submits review	Rejected	FR-25	API
TC-25	Promos	Admin creates coupon with usage limit 2; use it 3 times	Third use rejected	FR-28, 29	API
TC-26	Notify	Place an order	Confirmation email dispatched	FR-31	Integration
TC-27	Reports	Sales summary for a date range, by category	Correct totals	FR-33	API
TC-28	Reports	Low-stock report with threshold 40	Products below 40 listed	FR-34	API
TC-29	Reports	Export a report as CSV	Valid CSV file	FR-35	API
TC-30	Security	Submit script/NoSQL operator in a text field	Input sanitised; no execution	NFR-07	Security

7.3 Requirements Verification Matrix

Table 7.3: *Coverage of feature areas by test cases*

Feature area	Requirements	Test cases	Coverage
Authentication & access	FR-01 to FR-07	TC-01 to TC-07	7 of 7
Catalog & inventory	FR-08 to FR-13	TC-08 to TC-11, TC-16	6 of 6
Cart & checkout	FR-14 to FR-19	TC-12 to TC-17	6 of 6
Orders & fulfilment	FR-20 to FR-24	TC-16, TC-18 to TC-22	5 of 5
Reviews & ratings	FR-25 to FR-27	TC-23, TC-24	FR-27 via admin moderation check
Promotions	FR-28 to FR-30	TC-13, TC-14, TC-25	FR-30 via vendor discount check
Notifications	FR-31, FR-32	TC-18, TC-26	2 of 2
Reporting	FR-33 to FR-35	TC-27 to TC-29	3 of 3
Non-functional	NFR-01 to NFR-09	TC-06, TC-30, load and security suites	Planned in Increment 4

CHAPTER 8: RESULTS, USER INTERFACES & WORKFLOWS

This chapter presents the 26 prototype screens of the BlueCart interface. The screens are grouped by role. For each group, the text states which requirement the screen demonstrates and what the user can do on it.

8.1 Customer Authentication Screens

The **Login** screen takes an email or username and a password, offers a Remember-me option and a Forgot-password link (FR-02, FR-06), and shows inline validation under each field within 200 ms (SRS

3.1). The **Registration** screen collects name, email, mobile number and a confirmed password of at least eight characters (FR-01).

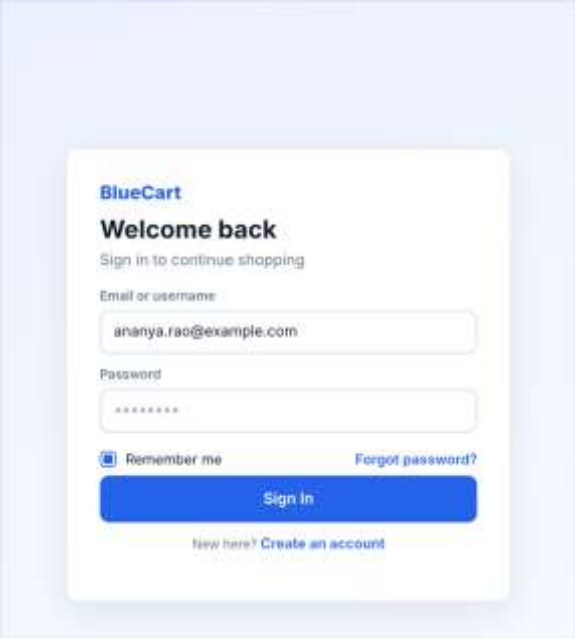


Figure 8.1: *Login page*

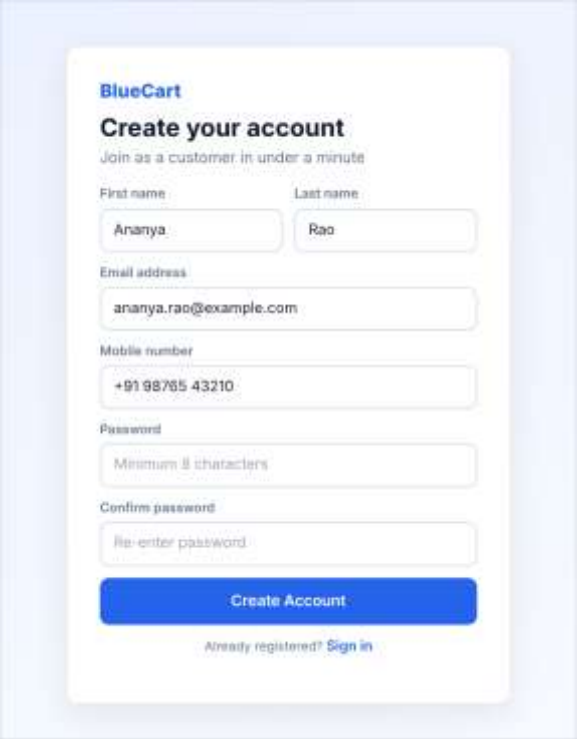


Figure 8.2: *Customer registration page*

8.2 Home, Category and Search

The **home page** carries the persistent top bar with search, wishlist, account and cart indicator (SRS 3.1), a promotional hero banner, category shortcuts and a Featured Products grid with discount badges, ratings and one-click Add to Cart.

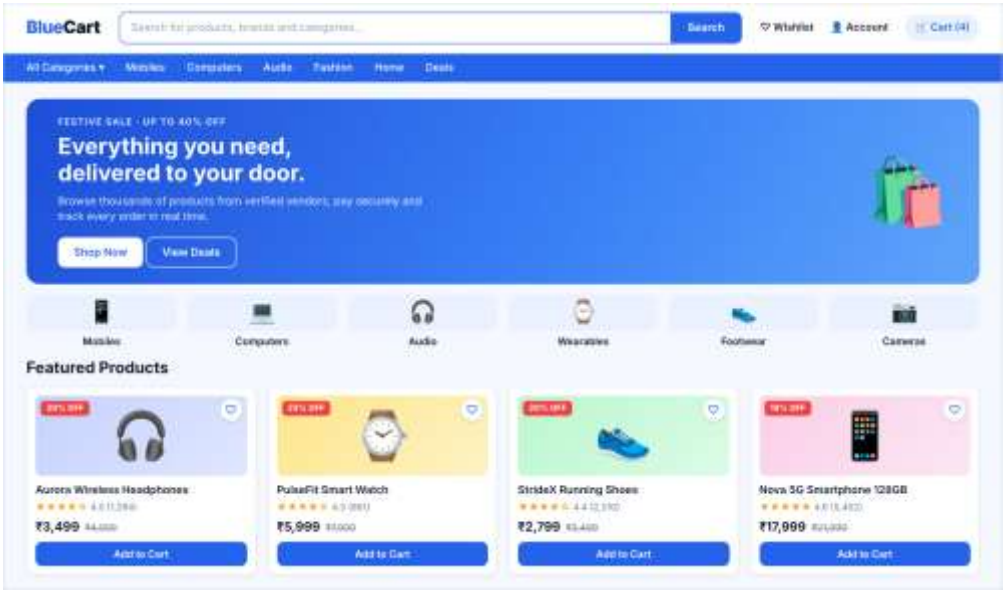


Figure 8.3: *Storefront home page with featured products*

The **Product Listing** page adds a filter panel for category, price range, customer rating and availability, a sort control and pagination, which together realise FR-10. The **Category** page applies the same pattern to a single department, and the **Search Results** page keeps the typed query visible and highlights matching items.

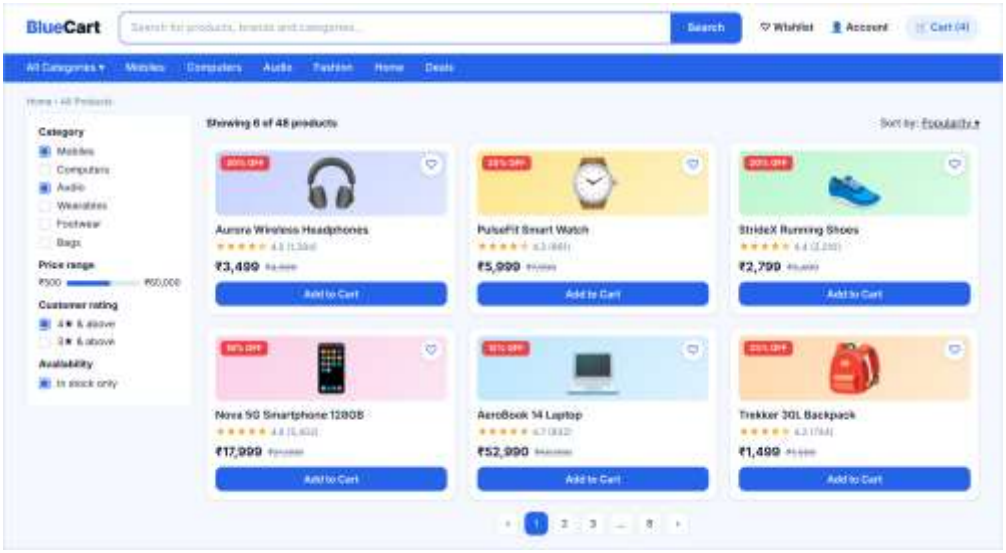


Figure 8.4: *Product listing with filters and sorting*

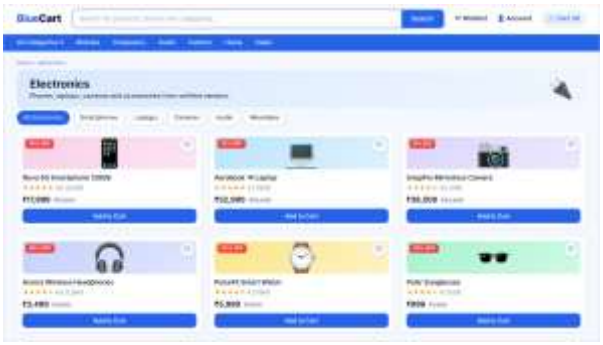


Figure 8.5: *Category page (Audio)*



Figure 8.6: *Search results page*

8.3 Product Details and Reviews

The **Product Details** page shows the gallery, price with discount, SKU (FR-09), stock indicator, vendor, delivery estimate and tabs for description, specifications and reviews. Quantity, Add to Cart, Buy Now and Wishlist actions are grouped beside the price. The **Ratings & Reviews** screen summarises the star distribution and lets a verified purchaser write a review (FR-25, FR-26).



Figure 8.7: *Product details page*

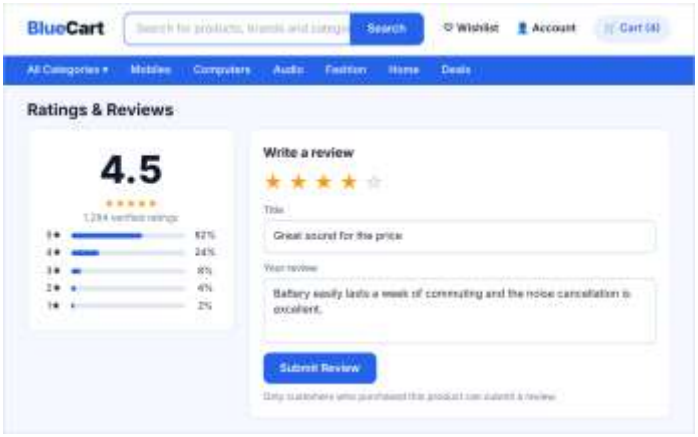


Figure 8.8: *Ratings and reviews page*

8.4 Cart and Wishlist

The **Shopping Cart** lists each line with quantity steppers, Remove and Move-to-wishlist links (FR-14, FR-15), and an order summary in which the customer can apply a coupon such as SAVE10 and see the discount, shipping charge and final total update at once (FR-16). The **Wishlist** keeps saved items so they can be moved to the cart later.

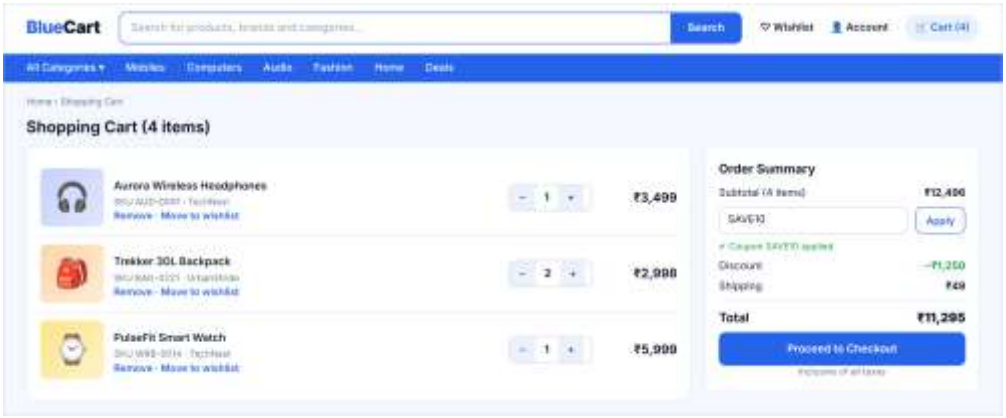


Figure 8.9: *Shopping cart with coupon and order summary*

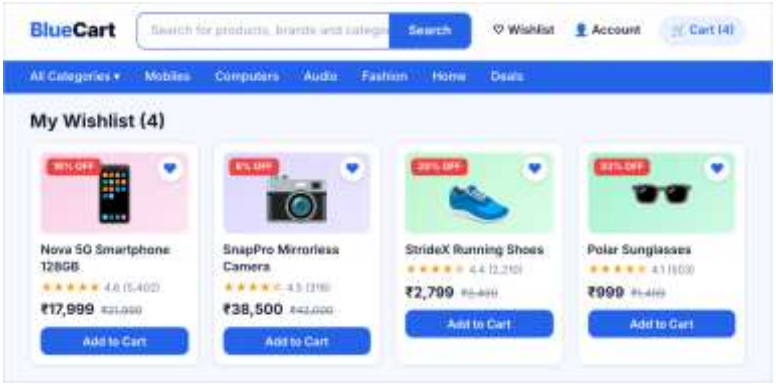


Figure 8.10: *Wishlist page*

8.5 Checkout, Payment and Confirmation

Checkout is a four-step flow (Cart, Address, Shipping, Payment) with a persistent order summary. The address page captures delivery details and offers standard and express delivery (FR-18). The **Payment** page hands card, UPI, net-banking or cash-on-delivery choices to the gateway and states that card data is never stored on the platform (FR-19, NFR-08). On success, the **Order Confirmation** page shows the unique order ID, amount paid, estimated delivery and a status timeline (FR-20, FR-31).

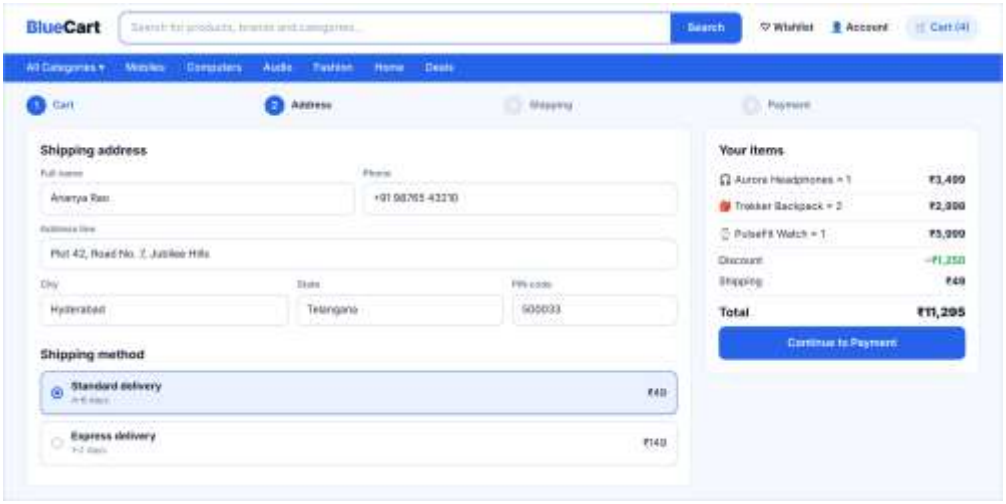


Figure 8.11: *Checkout: shipping address and delivery method*



Figure 8.12: *Secure payment page*

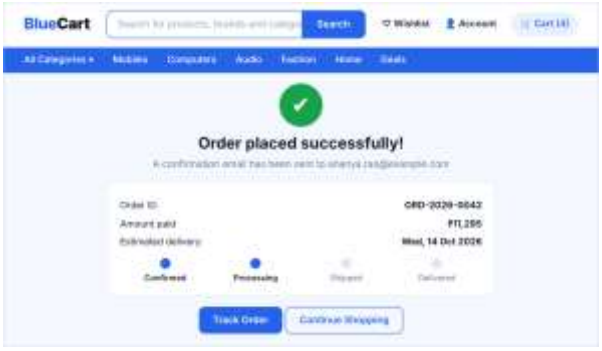


Figure 8.13: *Order confirmation page*

8.6 Orders, Profile and Information Pages

My Orders lists orders with a status pill and context actions (Cancel, Track, Return, Reorder) that are enabled only when the order state allows them (FR-22 to FR-24). The **Profile** page edits personal details, saved addresses, password and notification preferences (FR-32). The **Contact Us** and **About Us** pages provide support channels and a short description of the platform.

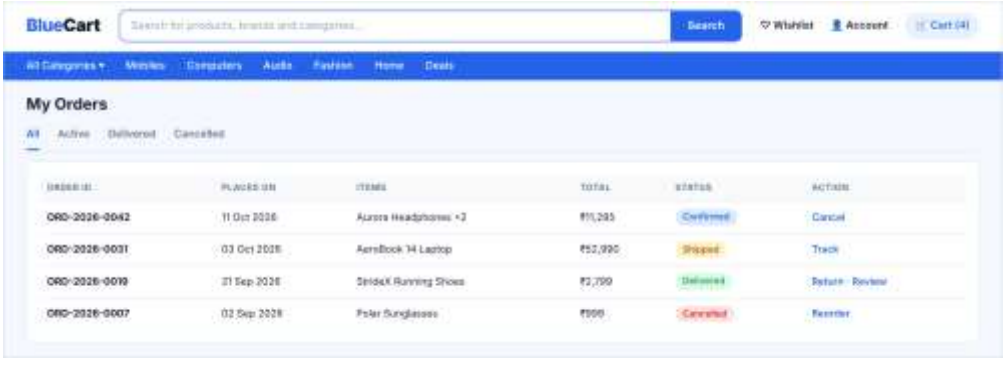


Figure 8.14: *My Orders page with order status*

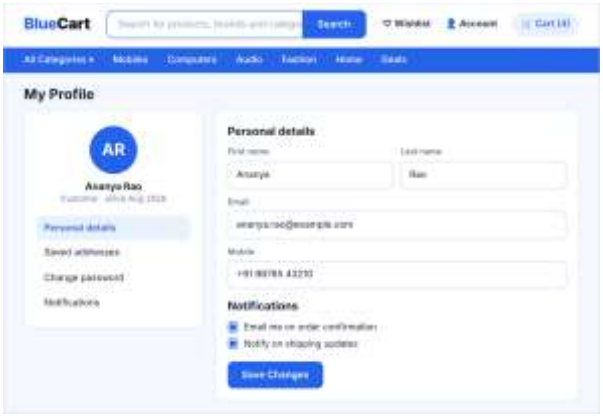


Figure 8.15: *User profile page*

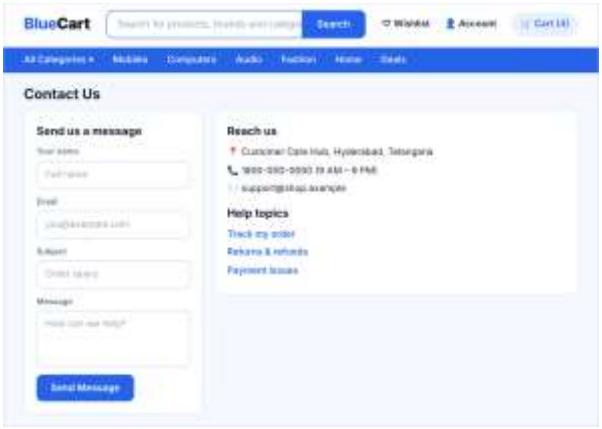


Figure 8.16: *Contact Us page*

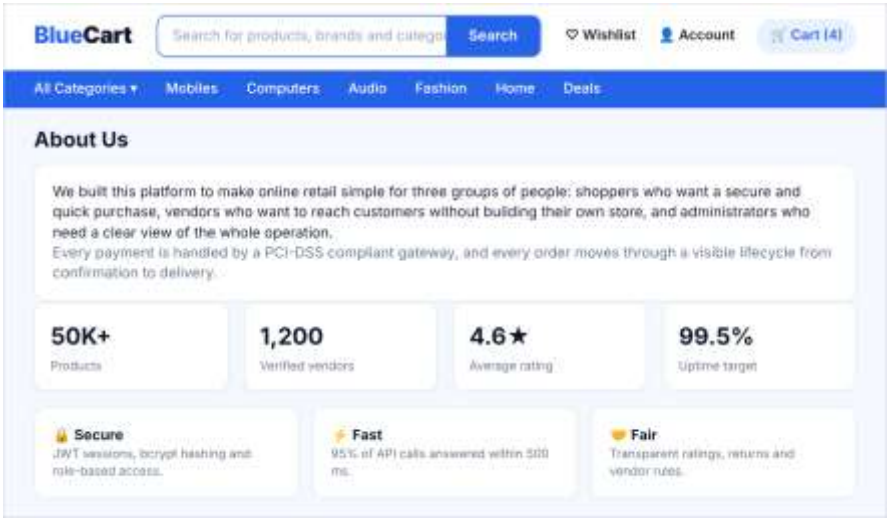


Figure 8.17: *About Us page*

8.7 Vendor Console

The **Vendor Console** uses a navy sidebar and shows orders to fulfil, shipped counts, active listings and earnings. A vendor changes an order's fulfilment status from a drop-down in the table (FR-21), and sees only their own catalogue (business rule 5.5).



Figure 8.18: *Vendor console with orders to fulfil*

8.8 Administrator Panel

Administrators sign in through a restricted **Admin Login** that warns that accounts lock after five failed attempts and that all actions are logged (FR-07, NFR-09). The **Dashboard** summarises revenue, orders, customers and products, a seven-day sales chart, low-stock alerts and recent orders.

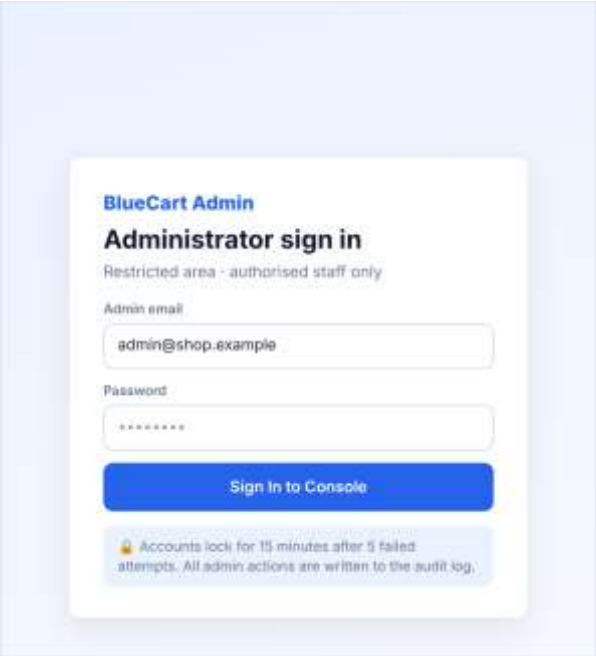


Figure 8.19: *Administrator login*



Figure 8.20: *Coupons and promotions*

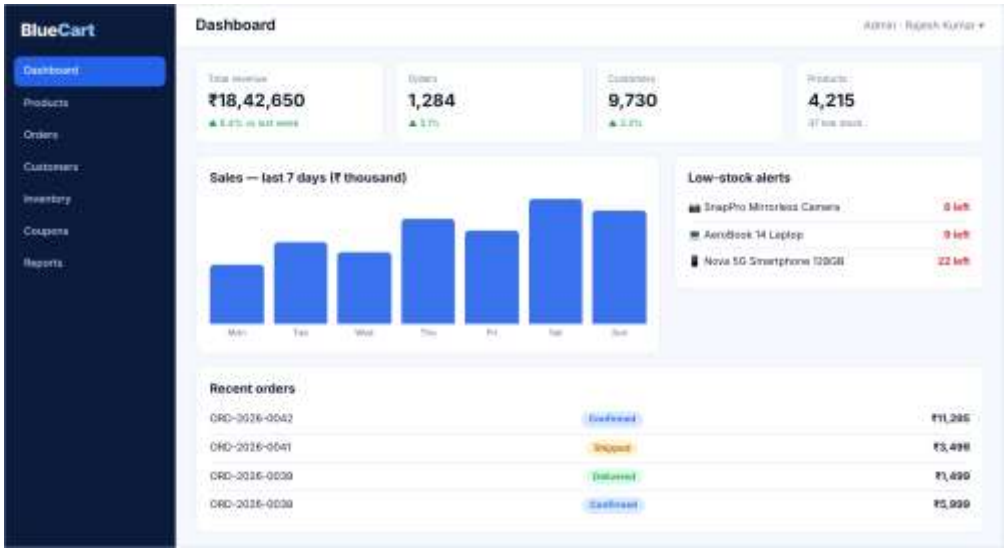


Figure 8.21: *Administrator dashboard*

Product Management supports search, category filter, add, edit and deactivate actions (FR-08, FR-12, FR-13). **Order Management** exposes tabs per status and a logged override action. **Customer Management** lists customers with account status such as Active, Locked or Suspended. **Inventory Management** highlights products below the configurable threshold and links to restock (FR-11, FR-34). **Reports & Analytics** provide date and vendor filters, monthly sales and category breakdowns with CSV export (FR-33, FR-35). **Coupons & Promotions** lists codes with discount, validity, usage and status (FR-28, FR-29).

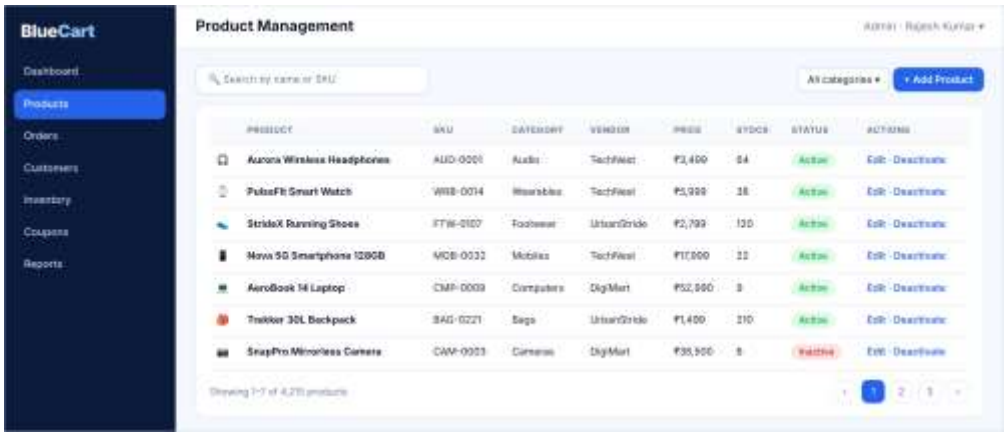


Figure 8.22: *Product management*

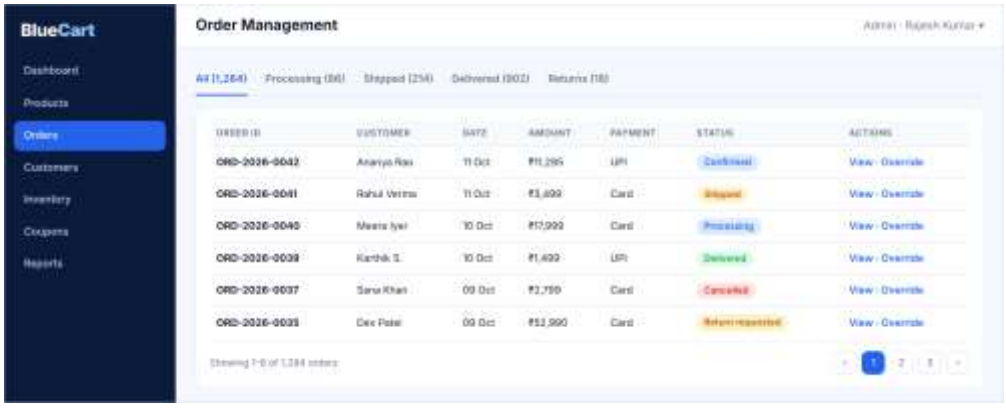


Figure 8.23: *Order management*

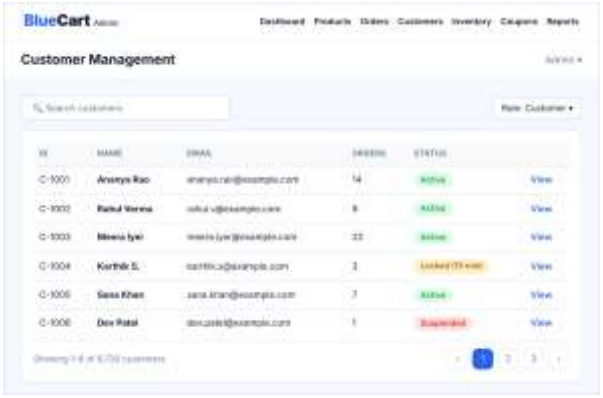


Figure 8.24: *Customer management*

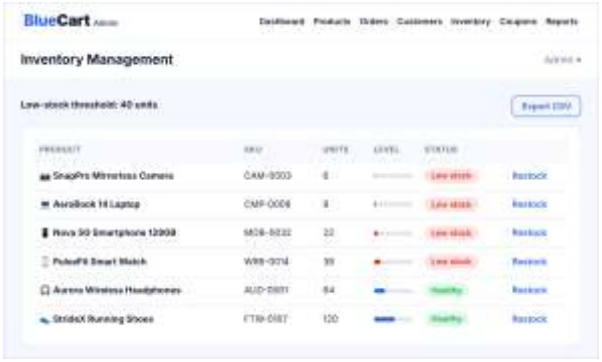


Figure 8.25: *Inventory management*



Figure 8.26: *Reports and analytics*

8.9 Summary of Results

Table 8.1: *Prototype coverage against the SRS*

Feature area	Screens	Status
Authentication and access	Login, Registration, Admin Login	Covered
Catalogue and inventory	Home, Listing, Category, Search, Details, Product Mgmt, Inventory	Covered
Cart and checkout	Cart, Wishlist, Checkout, Payment	Covered
Orders	Confirmation, My Orders, Order Mgmt, Vendor Console	Covered
Reviews and promotions	Reviews, Coupons	Covered
Notifications and profile	Profile, Confirmation notice	Covered
Reporting and administration	Dashboard, Customers, Reports	Covered
Information pages	Contact Us, About Us	Covered

All eight SRS feature areas are represented by at least one screen, and every one of the 35 functional requirements is linked to a screen in the requirements matrix of Chapter 3 and to a test case in Chapter 7. Core tasks such as adding a product to the cart need no more than three interactions (home → product → Add to Cart), meeting the usability target in SRS 5.4.

CHAPTER 9: ADVANTAGES, LIMITATIONS, FUTURE ENHANCEMENTS & CONCLUSION

9.1 Advantages

- One platform serves customers, vendors and administrators with strictly separated permissions.
- Server-side pricing and stock validation prevent tampering and overselling.
- Sensitive data is protected through bcrypt, TLS, input sanitisation and a PCI-DSS gateway.
- A stateless API and containerised back end allow horizontal scaling for sale events.
- Reports and CSV export give owners evidence for restocking and promotion decisions.

9.2 Limitations

- Version 1.0 supports one currency and the English language only.
- Shipment tracking is limited to vendor-updated statuses; live courier integration is future work.
- The prototype is a visual design; payment and notification flows need real gateway credentials to be exercised end to end.
- No multi-vendor commission or settlement logic is included (TBD-02).

9.3 Future Enhancements

- Courier API integration for real-time shipment tracking (TBD-05).
- Multi-currency and multi-language support (TBD-04).
- Vendor commission and settlement engine for a full multi-vendor marketplace.
- Recommendation engine and personalised home page based on browsing and purchase history.
- Mobile applications and WebSocket-based real-time order notifications.

9.4 Conclusion

This case study converted the Online Shopping System SRS into a complete engineering package: justified process model, feasibility analysis, architecture, UML and database designs, algorithms, a traceable test plan and a 26-screen interface prototype. The Incremental model fits the prioritised requirements, and the three-tier, stateless design satisfies the performance, security and scalability goals. The result gives a clear and testable blueprint from which the four planned increments can be implemented.

REFERENCES

[1] IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications, IEEE Computer Society.

[2] IEEE Std 1016-2009, IEEE Standard for Information Technology: Systems Design: Software Design Descriptions.

[3] Ian Sommerville, Software Engineering, 10th Edition, Pearson Education, 2016.

[4] Roger S. Pressman and Bruce R. Maxim, Software Engineering: A Practitioner's Approach, 9th Edition, McGraw-Hill, 2020.

[5] Node.js Foundation, Node.js Documentation, https://nodejs.org/en/docs

[6] MongoDB Inc., MongoDB Manual, https://www.mongodb.com/docs/manual/

[7] M. Jones, J. Bradley and N. Sakimura, JSON Web Token (JWT), RFC 7519, IETF, 2015.

[8] OWASP Foundation, OWASP Top Ten Web Application Security Risks, https://owasp.org/www-project-top-ten/

[9] PCI Security Standards Council, PCI DSS Quick Reference Guide, https://www.pcisecuritystandards.org/

[10] Vishnu Teja Karampuri, Software Requirements Specification for Online Shopping System (OSS), Version 1.0, CBIT Hyderabad, July 2026.